

# claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-a9087def0387fac88.jsonl`  
- SHA-256: `6ccc9a176e0c8c413115b600adb8e8d3738f57940e51f6ecc187167519df0180`  
- Source modified: `2026-07-03T17:35:45+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

## Transcript

### 1. user (2026-07-03T17:33:54.101Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch codex/a2-golden-manifest-portability (HEAD 5a91701). Diff: git diff origin/master..HEAD. New module backend/eval/golden\_manifest.py (378 lines) + consumer rewires.

ANGLE ? simplification. Flag unnecessary complexity the diff adds; name the simpler equivalent:

1. API surface: FIVE public functions (load\_manifest, resolve\_manifest\_paths, load\_resolved\_manifest, resolve\_manifest\_specs, resolve\_existing\_path) for what consumers use as ONE call (load\_resolved\_manifest). Are resolve\_manifest\_paths and resolve\_manifest\_specs both needed publicly (resolve\_manifest\_paths is a 2-line wrapper)? Is resolve\_existing\_path used anywhere outside the module?
2. \_candidate\_dirs/\_expand\_root: overlapping dir lists (root/'output' appears in both; manifest\_dir twice via candidates[0] raw + \_candidate\_dirs). Could the candidate generation be one flat comprehension? Is the field->dir indirection (\_FIELD\_DIR) earning its keep for exactly 2 fields?
3. Dead/unused parameters: does anything pass extra\_roots or repo\_root\_path outside tests/CLI? required\_fields used with a non-default anywhere besides rally\_seg\_eval?
4. ManifestIssue dataclass + message() ? used beyond error formatting? Could issues just be strings?
5. In consumers: any now-unneeded code left behind (e.g., per-file existence checks that the strict loader now makes redundant, leftover json imports, docstrings still describing the old behavior ? check rally\_seg\_eval.load\_golden docstring and test\_served\_gate\_a\_regression module docstring which described the OLD subset-filter semantics).

Only flag where the simpler form is clearly nameable. Return up to 6 candidates as JSON [{file, line, summary, failure\_scenario}]. [] if none. Final message = only the JSON array.

### 2. assistant (2026-07-03T17:33:55.905Z)

I'll analyze PR #452 for unnecessary complexity by examining the diff, the new module, and its consumers.

### 3. user (2026-07-03T17:33:56.884Z)

5a91701 eval: add portable golden manifest check

### 4. assistant (2026-07-03T17:33:58.664Z)

Let me fetch the diff and examine the new module and its consumers.

### 5. user (2026-07-03T17:34:00.551Z)

|                                 |  |      |
|---------------------------------|--|------|
| backend/eval/calibrate_local.py |  | 5 +- |
| backend/eval/distill_local.py   |  | 5 +- |
| backend/eval/fusion_golden.py   |  | 4 +- |

```

backend/eval/golden_manifest.py      | 378 ++++++
backend/eval/rally_seg_eval.py       | 4 +-
backend/eval/rally_seq_proto.py      | 5 +-
backend/eval/serve_contrast.py       | 4 +-
backend/eval/served_gate_a.py        | 5 +-
docs/ALPHA_LAUNCH_READINESS.md      | 11 +-
docs/data_pipeline/GOLDEN_VIDEOS.md | 8 +-
tests/test_fusion_compare.py         | 4 +
tests/test_fusion_golden_telemetry.py | 3 +
tests/test_golden_manifest.py        | 158 ++++++
tests/test_served_gate_a_regression.py | 21 +-
training/gen0/harness.py             | 4 +-
15 files changed, 586 insertions(+), 33 deletions(-)

```

## 6. user (2026-07-03T17:34:03.847Z)

```

1      """Portable golden-corpus manifest loader and smoke checker.
2
3      The owner-side golden manifest is intentionally gitignored and has historically
4      carried absolute paths from whichever box ingested a clip. This module keeps that
5      artifact usable after the workspace moves by resolving stale paths against the
6      current repo layout and failing loudly when required corpus files are absent.
7
8      Default local layout supported:
9      - ../Annotation Setup/Trajectories/<trajectory>.csv
10     - ../Annotation Setup/Golden Labelled/<labels>.csv
11     - output/<labels>.csv
12
13     Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14     """
15
16     from __future__ import annotations
17
18     import argparse
19     import json
20     import os
21     import sys
22     from dataclasses import dataclass
23     from pathlib import Path, PureWindowsPath
24     from typing import Any, Iterable, Sequence
25
26     DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27     SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28     DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30     _FIELD_DIR = {
31         "trajectory": "Trajectories",
32         "labels": "Golden Labelled",
33     }
34
35
36     @dataclass(frozen=True)
37     class ManifestIssue:
38         """One manifest path problem found by the resolver."""
39
40         name: str
41         field: str
42         path: str
43         reason: str
44
45         def message(self) -> str:
46             shown = f" {self.path!r}" if self.path else ""
47             return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49

```

```

50     class ManifestPathError(RuntimeError):
51         """Raised when a strict manifest load finds missing required files."""
52
53         def __init__(self, manifest_path: str | os.PathLike[str], issues:
Sequence[ManifestIssue]):
54             self.manifest_path = os.fspath(manifest_path)
55             self.issues = list(issues)
56             detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
57             more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12} more"
58             super().__init__(
59                 f"golden manifest path check failed for {self.manifest_path}: "
60                 f"{len(self.issues)} issue(s)\n{detail}{more}"
61             )
62
63
64     def repo_root() -> Path:
65         """Return the repository root for this module."""
66
67         return Path(__file__).resolve().parents[2]
68
69
70     def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
71         """Load the flat golden manifest list and validate its outer shape."""
72
73         path = Path(manifest_path)
74         with path.open(encoding="utf-8") as f:
75             data = json.load(f)
76             if not isinstance(data, list):
77                 raise ValueError(f"golden manifest must be a JSON list: {path}")
78             out: list[dict[str, Any]] = []
79             for i, row in enumerate(data):
80                 if not isinstance(row, dict):
81                     raise ValueError(f"golden manifest row {i} is not an object: {path}")
82                 out.append(dict(row))
83             return out
84
85
86     def resolve_manifest_paths(
87         manifest_path: str | os.PathLike[str],
88         *,
89         repo_root_path: str | os.PathLike[str] | None = None,
90         extra_roots: Iterable[str | os.PathLike[str]] = (),
91         required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
92     ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
93         """Load and rebase required path fields in a golden manifest.
94
95         Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
96         metadata but replace any resolved ``trajectory`` / ``labels`` path with the
97         existing local path. Missing required fields are reported in ``issues``.
98         """
99
100         manifest = load_manifest(manifest_path)
101         return resolve_manifest_specs(
102             manifest,
103             manifest_path,
104             repo_root_path=repo_root_path,
105             extra_roots=extra_roots,
106             required_fields=required_fields,
107         )
108
109
110     def load_resolved_manifest(
111         manifest_path: str | os.PathLike[str],
112         *,
113         repo_root_path: str | os.PathLike[str] | None = None,

```

```

114     extra_roots: Iterable[str | os.PathLike[str]] = (),
115     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
116     strict: bool = True,
117 ) -> list[dict[str, Any]]:
118     """Load a manifest with stale paths rebased; optionally raise on missing files."""
119
120     specs, issues = resolve_manifest_paths(
121         manifest_path,
122         repo_root_path=repo_root_path,
123         extra_roots=extra_roots,
124         required_fields=required_fields,
125     )
126     if strict and issues:
127         raise ManifestPathError(manifest_path, issues)
128     return specs
129
130
131 def resolve_manifest_specs(
132     specs: Sequence[dict[str, Any]],
133     manifest_path: str | os.PathLike[str],
134     *,
135     repo_root_path: str | os.PathLike[str] | None = None,
136     extra_roots: Iterable[str | os.PathLike[str]] = (),
137     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139     """Resolve an already-loaded manifest list."""
140
141     manifest = Path(manifest_path)
142     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
143     roots = [Path(p) for p in _env_roots()]
144     roots.extend(Path(p) for p in extra_roots)
145     required = tuple(required_fields)
146
147     resolved: list[dict[str, Any]] = []
148     issues: list[ManifestIssue] = []
149     for idx, spec in enumerate(specs):
150         row = dict(spec)
151         name = str(row.get("name") or f"row[{idx}]")
152         for field in required:
153             raw = row.get(field)
154             if not raw:
155                 issues.append(ManifestIssue(name, field, "", "required field missing"))
156                 continue
157             hit = resolve_existing_path(
158                 str(raw),
159                 field,
160                 manifest_path=manifest,
161                 repo_root_path=root,
162                 extra_roots=roots,
163             )
164             if hit is None:
165                 issues.append(ManifestIssue(name, field, str(raw), "file not found"))
166             else:
167                 row[field] = str(hit)
168             resolved.append(row)
169     return resolved, issues
170
171
172 def resolve_existing_path(
173     raw_path: str,
174     field: str,
175     *,
176     manifest_path: str | os.PathLike[str],
177     repo_root_path: str | os.PathLike[str],
178     extra_roots: Iterable[str | os.PathLike[str]] = (),

```

```

179 ) -> Path | None:
180     """Resolve one manifest path field to an existing file, if possible."""
181
182     for candidate in _candidate_paths(
183         raw_path,
184         field,
185         Path(manifest_path),
186         Path(repo_root_path),
187         [Path(p) for p in extra_roots],
188     ):
189         if candidate.is_file():
190             return candidate.resolve()
191     return None
192
193
194 def _env_roots() -> list[Path]:
195     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
196     if not raw:
197         return []
198     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
199
200
201 def _basename(raw_path: str) -> str:
202     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203     win_name = PureWindowsPath(raw_path).name
204     return win_name or Path(raw_path).name
205
206
207 def _candidate_paths(
208     raw_path: str,
209     field: str,
210     manifest_path: Path,
211     root: Path,
212     extra_roots: Sequence[Path],
213 ) -> list[Path]:
214     basename = _basename(raw_path)
215     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216     manifest_dir = manifest_path.resolve().parent
217     candidates: list[Path] = [raw]
218
219     if not raw.is_absolute():
220         candidates.extend([manifest_dir / raw, root / raw])
221
222     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223         candidates.append(directory / basename)
224     return _dedup_paths(candidates)
225
226
227 def _candidate_dirs(
228     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229 ) -> list[Path]:
230     field_dir = _FIELD_DIR.get(field)
231     dirs: list[Path] = [manifest_dir]
232     if field_dir:
233         dirs.extend(
234             [
235                 root.parent / "Annotation Setup" / field_dir,
236                 root.parent / "Annotation Setup" / "Collect" / field_dir,
237                 root / "output",
238             ]
239         )
240     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241     return _dedup_paths(dirs)
242
243

```

```

244 def _expand_root(root: Path, field: str) -> list[Path]:
245     field_dir = _FIELD_DIR.get(field)
246     dirs = [root, root / "output"]
247     if field_dir:
248         dirs.extend(
249             [
250                 root / field_dir,
251                 root / "Annotation Setup" / field_dir,
252                 root / "Annotation Setup" / "Collect" / field_dir,
253             ]
254         )
255     return dirs
256
257
258 def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259     out: list[Path] = []
260     seen: set[str] = set()
261     for path in paths:
262         try:
263             key = os.path.normcase(os.path.abspath(os.fspath(path)))
264         except OSError:
265             key = os.fspath(path)
266         if key in seen:
267             continue
268         seen.add(key)
269         out.append(path)
270     return out
271
272
273 def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
274     """Write a resolved manifest JSON atomically enough for local tooling."""
275
276     out = Path(path)
277     out.parent.mkdir(parents=True, exist_ok=True)
278     tmp = out.with_name(out.name + ".tmp")
279     with tmp.open("w", encoding="utf-8") as f:
280         json.dump(list(specs), f, indent=2)
281         f.write("\n")
282     os.replace(tmp, out)
283
284
285 def build_parser() -> argparse.ArgumentParser:
286     p = argparse.ArgumentParser(
287         description="Check/rebase the local golden corpus manifest paths."
288     )
289     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
290     p.add_argument(
291         "--repo-root",
292         default=None,
293         help="repo root used for default sibling output/Annotation Setup lookups",
294     )
295     p.add_argument(
296         "--root",
297         action="append",
298         default=[],
299         help=(
300             "additional search root; repeatable. Also accepts roots via "
301             f"{{SEARCH_ROOTS_ENV}} separated by os.pathsep."
302         ),
303     )
304     p.add_argument(
305         "--require",
306         action="append",
307         choices=sorted(_FIELD_DIR),

```

```

308         default=None,
309         help="path field to require; repeatable. Defaults to trajectory and labels.",
310     )
311     p.add_argument(
312         "--min-count",
313         type=int,
314         default=None,
315         help="fail unless at least this many manifest rows are present",
316     )
317     p.add_argument(
318         "--write",
319         default=None,
320         help="optional path to write the resolved local manifest JSON",
321     )
322     return p
323
324
325 def main(argv: Sequence[str] | None = None) -> int:
326     args = build_parser().parse_args(argv)
327     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328     try:
329         original = load_manifest(args.manifest)
330         resolved, issues = resolve_manifest_specs(
331             original,
332             args.manifest,
333             repo_root_path=args.repo_root,
334             extra_roots=args.root,
335             required_fields=required,
336         )
337     except (OSError, ValueError, json.JSONDecodeError) as e:
338         print(f"ERROR: {e}", file=sys.stderr)
339         return 2
340
341     if args.min_count is not None and len(resolved) < args.min_count:
342         issues.append(
343             ManifestIssue(
344                 "manifest",
345                 "rows",
346                 str(len(resolved)),
347                 f"expected at least {args.min_count} row(s)",
348             )
349         )
350
351     if issues:
352         print(
353             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354             file=sys.stderr,
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)
358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "
368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)

```

```

373         print(f"wrote resolved manifest: {args.write}")
374     return 0
375
376
377     if __name__ == "__main__":
378         raise SystemExit(main())
379

```

## 7. assistant (2026-07-03T17:34:05.779Z)

Now let me check the consumers to see how they use this module:

## 8. user (2026-07-03T17:34:07.644Z)

```

diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
 from __future__ import annotations

 import argparse
-import json
 from typing import Callable, Dict, List, Tuple

 from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.pipeline.detectors import rally_gate
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, metric=args.metric)

diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
 from __future__ import annotations

 import argparse
-import json
 from typing import Dict, List, Optional, Tuple

 from backend.eval import windowing as _windowing
 from backend.eval.calibrate_wasb import load_golden_gts
 from backend.eval.classifier import LogisticRegression
 from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import score
 from backend.eval.training import label_candidates
 from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
 )

```



```

    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
-   with open(args.manifest, encoding="utf-8") as f:
-       specs = json.load(f)
+   specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)

diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

    from backend.eval import distill_local
    from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.training import label_candidates
    from backend.pipeline.detectors import fusion_features as ff
    from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
    fresh: bool = False,
    smallest_first: bool = False,
) -> List[str]:
-   with open(manifest_path, encoding="utf-8") as f:
-       specs = json.load(f)
+   specs = load_resolved_manifest(manifest_path)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 0acca2d..3846ce0 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
    from typing import Any, Dict, List, Optional, Tuple

    from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.metrics import Interval
    from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

@@ -185,8 +186,7 @@ def load_golden(
) -> List[Dict[str, Any]]:
    """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
-   with open(manifest_path) as fh:
-       manifest = json.load(fh)
+   manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
    out: List[Dict[str, Any]] = []
    for v in manifest:
        feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
index 2ea9483..b017f94 100644
--- a/backend/eval/rally_seq_proto.py
+++ b/backend/eval/rally_seq_proto.py
@@ -29,7 +29,6 @@ Run:
    from __future__ import annotations

    import argparse
-import json
    from typing import Dict, List, Tuple

```

```

import numpy as np
@@ -38,6 +37,7 @@ from scipy.stats import rankdata

from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score

# Feature extraction is single-sourced in backend/features (ADR-008): training and
@@ -196,8 +196,7 @@ def main() -> None:
    help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
    )
    args = ap.parse_args()
-   with open(args.manifest, encoding="utf-8") as f:
-       run(json.load(f))
+   run(load_resolved_manifest(args.manifest))

if __name__ == "__main__":
diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
index 328857b..0f1b1f4 100644
--- a/backend/eval/serve_contrast.py
+++ b/backend/eval/serve_contrast.py
@@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple
from backend.eval import windowing as W
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score
from backend.eval.promotion import promote_if_served_safe
from backend.pipeline.detectors import rally_gate
@@ -194,8 +195,7 @@ def main() -> None:
    )
    args = ap.parse_args()

-   with open(args.manifest, encoding="utf-8") as f:
-       specs = json.load(f)
+   specs = load_resolved_manifest(args.manifest)
    result = run_contrast(
        specs,
        iou=args.iou,
diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
index 994b9c1..c2151fb 100644
--- a/backend/eval/served_gate_a.py
+++ b/backend/eval/served_gate_a.py
@@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
from __future__ import annotations

import argparse
-import json
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
from backend.eval import segmentation_metrics as _segm
from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import Interval, Score, score
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
@@ -196,8 +196,7 @@ def format_table(
def run(
    manifest_path: str, iou: float = 0.5
) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
-   with open(manifest_path, encoding="utf-8") as f:

```

```

-         specs = json.load(f)
+     specs = load_resolved_manifest(manifest_path)
+     results = evaluate_manifest(specs, iou=iou)
+     return results, aggregate(results)

diff --git a/tests/test_served_gate_a_regression.py b/tests/test_served_gate_a_regression.py
index 7fd7630..56848df 100644
--- a/tests/test_served_gate_a_regression.py
+++ b/tests/test_served_gate_a_regression.py
@@ -37,7 +37,6 @@ re-measurement as the corpus grows.

```

```

from __future__ import annotations

import json
import os

import pytest
@@ -49,6 +48,10 @@ pytest.importorskip("cv2")
from backend.eval import segmentation_metrics as _segm # noqa: E402
from backend.eval import served_gate_a as sga # noqa: E402
from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
+from backend.eval.golden_manifest import ( # noqa: E402
+    ManifestPathError,
+    load_resolved_manifest,
+)
from backend.eval.metrics import score # noqa: E402
from backend.pipeline.detectors import rally_gate # noqa: E402
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
@@ -64,18 +67,14 @@ MANIFEST = os.environ.get(

def _golden_specs():
-    """Return the manifest specs whose trajectory + label files are all present, else []."""
+    """Return rebased manifest specs when trajectory + label files are present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
-    with open(MANIFEST, encoding="utf-8") as f:
-        specs = json.load(f)
-    ok = [
-        s
-        for s in specs
-        if os.path.isfile(str(s.get("trajectory", "")))
-        and os.path.isfile(str(s.get("labels", "")))
-    ]
-    return ok if len(ok) >= 6 else []
+    try:
+        specs = load_resolved_manifest(MANIFEST)
+    except ManifestPathError:
+        return []
+    return specs if len(specs) >= 6 else []

```

```

_SPECS = _golden_specs()
diff --git a/training/gen0/harness.py b/training/gen0/harness.py
index 6eec29f..433d2cb 100644
--- a/training/gen0/harness.py
+++ b/training/gen0/harness.py
@@ -36,6 +36,7 @@ from typing import Dict, List, Optional
import numpy as np

from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.gt_loader import load_gt_intervals
from backend.eval.rally_seq_proto import (
    _seg_f1,

```

```

@@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
def run(
    manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
) -> dict:
-   with open(manifest_path, encoding="utf-8") as f:
-       specs = json.load(f)
+   specs = load_resolved_manifest(manifest_path)
    floor = None
    if floor_path:
        with open(floor_path, encoding="utf-8") as f:

```

## 9. assistant (2026-07-03T17:34:10.901Z)

Let me look at the test file to understand what the module does:

## 10. user (2026-07-03T17:34:11.564Z)

```

1     import json
2     from pathlib import Path
3
4     import pytest
5
6     from backend.eval import golden_manifest as gm
7
8
9     def _write(path: Path, body: str = "\n") -> Path:
10         path.parent.mkdir(parents=True, exist_ok=True)
11         path.write_text(body, encoding="utf-8")
12         return path
13
14
15     def _write_manifest(repo: Path, rows: list[dict]) -> Path:
16         path = repo / "output" / "human_lovo_manifest.json"
17         path.parent.mkdir(parents=True, exist_ok=True)
18         path.write_text(json.dumps(rows), encoding="utf-8")
19         return path
20
21
22     def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23         workspace = tmp_path / "workspace"
24         repo = workspace / "badminton-highlight-indexer"
25         repo.mkdir(parents=True)
26         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27         old_label = _write(
28             workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29         )
30         output_label = _write(repo / "output" / "Bar.rallies.csv")
31         manifest = _write_manifest(
32             repo,
33             [
34                 {
35                     "name": "foo",
36                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
37                     "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
38                     "fps": 30,
39                     "frame_width": 1920,
40                 },
41                 {
42                     "name": "bar",
43                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
44                     "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
45                     "fps": 30,

```

```

46         "frame_width": 1920,
47     },
48 ],
49 )
50
51 specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
52
53 assert issues == []
54 assert Path(specs[0]["trajectory"]) == traj.resolve()
55 assert Path(specs[0]["labels"]) == old_label.resolve()
56 assert Path(specs[1]["labels"]) == output_label.resolve()
57
58
59 def test_strict_loader_reports_missing_required_files(tmp_path):
60     repo = tmp_path / "repo"
61     repo.mkdir()
62     manifest = _write_manifest(
63         repo,
64         [
65             {
66                 "name": "missing",
67                 "trajectory": "C:/old/missing_traj.csv",
68                 "labels": "C:/old/missing_rallies.csv",
69             }
70         ],
71     )
72
73     with pytest.raises(gm.ManifestPathError) as excinfo:
74         gm.load_resolved_manifest(manifest, repo_root_path=repo)
75
76     assert len(excinfo.value.issues) == 2
77     assert "file not found" in str(excinfo.value)
78
79
80 def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
81     workspace = tmp_path / "workspace"
82     repo = workspace / "badminton-highlight-indexer"
83     repo.mkdir(parents=True)
84     traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
85     manifest = _write_manifest(
86         repo,
87         [
88             {
89                 "name": "foo",
90                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
91                 "labels": "C:/old/missing_rallies.csv",
92             }
93         ],
94     )
95
96     specs, issues = gm.resolve_manifest_paths(
97         manifest, repo_root_path=repo, required_fields=("trajectory",)
98     )
99
100     assert issues == []
101     assert Path(specs[0]["trajectory"]) == traj.resolve()
102     assert specs[0]["labels"] == "C:/old/missing_rallies.csv"
103
104
105 def test_cli_writes_resolved_manifest(tmp_path, capsys):
106     workspace = tmp_path / "workspace"
107     repo = workspace / "badminton-highlight-indexer"
108     repo.mkdir(parents=True)
109     _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
110     _write(repo / "output" / "Foo.rallies.csv")

```

```

111     manifest = _write_manifest(
112         repo,
113         [
114             {
115                 "name": "foo",
116                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
117                 "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
118             }
119         ],
120     )
121     out = repo / "output" / "human_lovo_manifest.local.json"
122
123     rc = gm.main(
124         [
125             "--manifest",
126             str(manifest),
127             "--repo-root",
128             str(repo),
129             "--min-count",
130             "1",
131             "--write",
132             str(out),
133         ]
134     )
135
136     captured = capsys.readouterr()
137     assert rc == 0
138     assert "OK:" in captured.out
139     assert out.is_file()
140     resolved = json.loads(out.read_text(encoding="utf-8"))
141     assert Path(resolved[0]["trajectory"]).is_file()
142     assert Path(resolved[0]["labels"]).is_file()
143
144
145     def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
146         repo = tmp_path / "repo"
147         repo.mkdir()
148         manifest = _write_manifest(
149             repo,
150             [{"name": "foo", "trajectory": "missing.csv", "labels": "missing.rallies.csv"}],
151         )
152
153         rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
154
155         captured = capsys.readouterr()
156         assert rc == 2
157         assert "golden manifest path check failed" in captured.err
158         assert "foo: trajectory" in captured.err
159

```

#### 11. assistant (2026-07-03T17:34:13.485Z)

Now let me check if any of these functions are used outside the module in ways that justify their public exposure:

#### 12. user (2026-07-03T17:34:14.075Z)

Found 2 files  
tests\test\_golden\_manifest.py  
backend\eval\golden\_manifest.py

#### 13. user (2026-07-03T17:34:16.169Z)

Found 1 file

backend\eval\golden\_manifest.py

#### 14. user (2026-07-03T17:34:18.003Z)

Found 3 files

tests\test\_golden\_manifest.py  
backend\eval\rally\_seg\_eval.py  
backend\eval\golden\_manifest.py

#### 15. assistant (2026-07-03T17:34:19.703Z)

Let me check the actual usage in rally\_seg\_eval.py more closely:

#### 16. user (2026-07-03T17:34:20.440Z)

```
185     manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
186 ) -> List[Dict[str, Any]]:
187     """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
188     intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
flagged."""
189     manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
190     out: List[Dict[str, Any]] = []
191     for v in manifest:
192         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
193         gts: List[Interval] = []
194         if os.path.exists(feat):
```

#### 17. assistant (2026-07-03T17:34:24.161Z)

Now let me check if ManifestIssue is used beyond error formatting:

#### 18. user (2026-07-03T17:34:24.365Z)

Found 1 file

backend\eval\golden\_manifest.py

#### 19. assistant (2026-07-03T17:34:26.388Z)

Let me check if there are any remaining json imports in consumers that are now unneeded:

#### 20. user (2026-07-03T17:34:27.596Z)

```
diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

from backend.eval import distill_local
from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.training import label_candidates
from backend.pipeline.detectors import fusion_features as ff
from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
    fresh: bool = False,
    smallest_first: bool = False,
) -> List[str]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
```

# the LOVO result is order-independent. Proxy size by the trajectory CSV (one row/frame).

#### 21. user (2026-07-03T17:34:29.723Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17
18      from __future__ import annotations
19
20      import argparse
21      import json
22      import os
23      import time
24      from typing import Any, Dict, List, Optional
25
26      from backend.eval import distill_local
27      from backend.eval.calibrate_wasb import load_golden_gts
28      from backend.eval.golden_manifest import load_resolved_manifest
29      from backend.eval.training import label_candidates
30      from backend.pipeline.detectors import fusion_features as ff
```

#### 22. assistant (2026-07-03T17:34:31.678Z)

Let me check how json is used in fusion\_golden.py:

#### 23. user (2026-07-03T17:34:32.031Z)

```
215:      json.dump(data, f, indent=2)
```

#### 24. assistant (2026-07-03T17:34:33.952Z)

Good, json is still used in fusion\_golden.py. Let me check the other consumers to see if json is now unneeded:

#### 25. user (2026-07-03T17:34:35.540Z)

```
import json
```

#### 26. user (2026-07-03T17:34:38.470Z)

(Bash completed with no output)



## 27. assistant (2026-07-03T17:34:40.157Z)

So json was removed from calibrate\_local.py. Let me check if this is truly unused:

## 28. user (2026-07-03T17:34:41.476Z)

```
1      """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2      on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3      100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5      Pipeline being tuned (all local, no Gemini at runtime):
6          WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                               -> rally_gate (min_crossings)           -> rally windows
8
9      Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10     videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11     the honest "how precise is the local pipeline on an unseen video" number. The
12     output is a single local config + the precision it buys, with NO Gemini dependency
13     shipped.
14
15     Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16     calibration.{select_best, leave_one_video_out, improvement_report}.
17
18     Run:
19         python -m backend.eval.calibrate_local --manifest videos.json
20     where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21     (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22     """
23
24     from __future__ import annotations
25
26     import argparse
27     from typing import Callable, Dict, List, Tuple
28
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.calibration import (
31         evaluate,
32         improvement_report,
33         leave_one_video_out,
34         select_best,
35     )
36     from backend.eval.golden_manifest import load_resolved_manifest
37     from backend.pipeline.detectors import rally_gate
38     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
39
40     # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
41     LocalConfig = Tuple[float, float, float, int]
42     BASELINE_LOCAL: LocalConfig = (
43         0.02,
44         2.0,
45         2.0,
46         0,
47     ) # today's local default (windowing, no gate)
48
49
50     def make_local_predict_fn(
```

## 29. assistant (2026-07-03T17:34:43.494Z)

Now let me look at the docstring changes in test\_served\_gate\_a\_regression.py:

## 30. user (2026-07-03T17:34:43.855Z)

```

38     from __future__ import annotations
39
40     import os
41
42     import pytest
43
44     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
45     # (CI), skip the whole module rather than erroring at import.
46     pytest.importorskip("cv2")
47
48     from backend.eval import segmentation_metrics as _segm # noqa: E402
49     from backend.eval import served_gate_a as sga # noqa: E402
50     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
51     from backend.eval.golden_manifest import ( # noqa: E402
52         ManifestPathError,
53         load_resolved_manifest,
54     )
55     from backend.eval.metrics import score # noqa: E402
56     from backend.pipeline.detectors import rally_gate # noqa: E402
57     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
58
59     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
60     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
61     # out-of-tree checkout.
62     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
63     MANIFEST = os.environ.get(
64         "SERVED_GATE_A_MANIFEST",
65         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
66     )
67
68
69     def _golden_specs():
70         """Return rebased manifest specs when trajectory + label files are present, else
[ ]."""
71         if not os.path.isfile(MANIFEST):
72             return []
73         try:
74             specs = load_resolved_manifest(MANIFEST)
75         except ManifestPathError:
76             return []
77         return specs if len(specs) >= 6 else []
78
79
80     _SPECS = _golden_specs()
81     _skip = pytest.mark.skipif(
82         not _SPECS,
83         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
84     )
85
86
87     # ---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there,
not cv2) ----
88     def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
89         """The served litmus must pass a TYPED IndexingConfig (what production passes), not
a dict ?
90         `_enrich_candidate_stats` reads `idx_cfg.fusion`, so a dict raised AttributeError
(#398). Build
91         the config + drive the full served seam path on a synthetic trajectory (no golden
data needed)."""
92         from backend.config.models import IndexingConfig
93
94         cfg = sga._served_idx_cfg(2)

```

```

95         assert isinstance(cfg, IndexingConfig)
96         assert cfg.fusion.min_crossings == 2
97         traj = tmp_path / "synth.csv"
98         rows = ["Frame,Visibility,X,Y"]
99         for i in range(120): # oscillate across the net so windows + crossings form
100             x = 200 + (i % 20) * 40
101             rows.append(f"{i},1,{float(x)},300.0")
102         traj.write_text("\n".join(rows))
103         # The full served path: construct FusionSegmenter + _enrich_candidate_stats +
104         _select_for_handoff.
105         out = sga.served_handoff_windows(str(traj), fps=30.0, frame_width=1280.0,
106 min_crossings=2)
107         assert isinstance(out, list) # no AttributeError from the dict-vs-typed-config
108         regression
109         # ----- served-path
110         litmus
111         @_skip
112         def test_served_path_runs_on_all_golden_videos():
113             """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
114 EVERY
115             present golden trajectory and yields a handoff window list for each, for both gate
116 settings.
117             Asserts against the present corpus count (>= 6), not a stale fixed 6 ? the golden
118 set grows."""
119             results = sga.evaluate_manifest(_SPECS)

```

### 31. assistant (2026-07-03T17:34:45.926Z)

Now let me check the original docstring for \_golden\_specs:

### 32. user (2026-07-03T17:34:47.237Z)

```

    "SERVED_GATE_A_MANIFEST",
    os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
)

def _golden_specs():
    """Return the manifest specs whose trajectory + label files are all present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
    with open(MANIFEST, encoding="utf-8") as f:
        specs = json.load(f)
    ok = [
        s
        for s in specs
        if os.path.isfile(str(s.get("trajectory", "")))
        and os.path.isfile(str(s.get("labels", "")))
    ]
    return ok if len(ok) >= 6 else []

_SPECS = _golden_specs()
_skip = pytest.mark.skipif(
    not _SPECS,
    reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
)

```

### 33. assistant (2026-07-03T17:34:49.982Z)

Good catch! The docstring has changed but now describes what the function does (rebased manifest specs), not what was done before (filtering by per-file existence checks that are now done by load\_resolved\_manifest). Let me also check the module docstring:

#### 34. user (2026-07-03T17:34:50.310Z)

```
1      """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3      The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4      ``min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5      MagicMock runner). The offline ablation litmus
6      (`test_ablation.test_litmus_reproduces_gate_a`)
7      validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8      (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9      runs the real ``FusionSegmenter`` decision seams (`_enrich_candidate_stats` net-
crossings +
10     `_select_for_handoff` Gate A) over windows from the **production**
11     ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
12     the served path's measured behaviour.
13
14     GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
15     CI where the golden trajectories are absent; runs on the owner box where they are
present, over
16     **whatever golden clips are currently on disk** (`_golden_specs` requires >= 6).
17
18     CORPUS NOTE ? re-grounded 2026-06-22 (the golden set is mutable and grows over time;
this is the
19     SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has
since
20     outgrown). The original 6 *single-court* golden clips this litmus was first calibrated
on
21     (2026-06-14) have been retired from the box ? their labels are gone ? so the present
golden set is
22     the **15 multicourt clips** now on disk (mahadevpura_1/2/singles, GX0101xx, GX0x0094,
23     Badminton_BXH_2, Boxhill_Doubles, adarsh_avi_singles, kushagra_singles,
largetest_doubles, gbaaddy,
24     testlarge_short). On that corpus the served path is **recall-starved**: WASB tracks a
single shuttle
25     while several courts rally at once, so ``trajectory_to_action_windows`` emits FEWER
windows than
26     there are rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A
only ever
27     DROPS windows, on this under-segmented corpus it is mildly **F1-negative** (mean served
?F1 ?
28     -0.023), not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL
windowing the
29     same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation
(1.36?0.79).
30
31     The canonical **~+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
32     and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
33     (`docs/archives/quality-iterations/2026-06-first-ab-experiment/`). These tests now pin
the
34     **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
35     litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
36     re-measurement as the corpus grows.
```

```
36     """
37
38     from __future__ import annotations
39
40     import os
```

### 35. assistant (2026-07-03T17:34:53.323Z)

The module docstring doesn't describe the old per-file filtering semantics, so it's OK. Now let me check the `_candidate_dirs` and `_expand_root` to look for the overlapping logic:

### 36. user (2026-07-03T17:34:54.342Z)

```
227     def _candidate_dirs(
228         field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229     ) -> list[Path]:
230         field_dir = _FIELD_DIR.get(field)
231         dirs: list[Path] = [manifest_dir]
232         if field_dir:
233             dirs.extend(
234                 [
235                     root.parent / "Annotation Setup" / field_dir,
236                     root.parent / "Annotation Setup" / "Collect" / field_dir,
237                     root / "output",
238                 ]
239             )
240         dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241         return _dedup_paths(dirs)
242
243
244     def _expand_root(root: Path, field: str) -> list[Path]:
245         field_dir = _FIELD_DIR.get(field)
246         dirs = [root, root / "output"]
247         if field_dir:
248             dirs.extend(
249                 [
250                     root / field_dir,
251                     root / "Annotation Setup" / field_dir,
252                     root / "Annotation Setup" / "Collect" / field_dir,
253                 ]
254             )
255         return dirs
256
```

### 37. assistant (2026-07-03T17:34:56.333Z)

Now let me look for usage to understand if these parameters are actually used in the public API:

### 38. user (2026-07-03T17:34:57.572Z)

(Bash completed with no output)

### 39. assistant (2026-07-03T17:35:00.253Z)

Let me check the test file more thoroughly to see what parameters are actually used:

### 40. user (2026-07-03T17:35:01.523Z)

```
Exit code 2
grep: C:\Users\avidu\Projects\khe\skhelsutra-gurubadminton-highlight-indexerteststest_golden_manifest.py:
No such file or directory
```

### 41. user (2026-07-03T17:35:04.579Z)

```
51: specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
74: gm.load_resolved_manifest(manifest, repo_root_path=repo)
96: specs, issues = gm.resolve_manifest_paths(
```

#### 42. assistant (2026-07-03T17:35:07.203Z)

Let me look at `resolve_manifest_paths` more carefully - it's a 2-line wrapper:

#### 43. user (2026-07-03T17:35:07.656Z)

```
86 def resolve_manifest_paths(
87     manifest_path: str | os.PathLike[str],
88     *,
89     repo_root_path: str | os.PathLike[str] | None = None,
90     extra_roots: Iterable[str | os.PathLike[str]] = (),
91     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
92 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
93     """Load and rebase required path fields in a golden manifest.
94
95     Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
96     metadata but replace any resolved ``trajectory`` / ``labels`` path with the
97     existing local path. Missing required fields are reported in ``issues``.
98     """
99
100     manifest = load_manifest(manifest_path)
101     return resolve_manifest_specs(
102         manifest,
103         manifest_path,
104         repo_root_path=repo_root_path,
105         extra_roots=extra_roots,
106         required_fields=required_fields,
107     )
108
```

#### 44. assistant (2026-07-03T17:35:09.617Z)

Perfect. Now let me verify that `load_resolved_manifest` is the only public function actually used by consumers:

#### 45. user (2026-07-03T17:35:11.062Z)

```
backend/eval/calibrate_local.py:36:from backend.eval.golden_manifest import
load_resolved_manifest
backend/eval/distill_local.py:33:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/fusion_golden.py:28:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/rally_seg_eval.py:30:from backend.eval.golden_manifest import
load_resolved_manifest
backend/eval/rally_seq_proto.py:40:from backend.eval.golden_manifest import
load_resolved_manifest
backend/eval/served_gate_a.py:32:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/serve_contrast.py:35:from backend.eval.golden_manifest import
load_resolved_manifest
training/gen0/harness.py:39:from backend.eval.golden_manifest import load_resolved_manifest
```

#### 46. user (2026-07-03T17:35:16.591Z)

```
tests/test_served_gate_a_regression.py:51:from backend.eval.golden_manifest import ( # noqa:
E402
```

#### 47. user (2026-07-03T17:35:19.743Z)

```
from backend.eval.golden_manifest import ( # noqa: E402
    ManifestPathError,
    load_resolved_manifest,
```

```
)
from backend.eval.metrics import score # noqa: E402
from backend.pipeline.detectors import rally_gate # noqa: E402
```

#### 48. assistant (2026-07-03T17:35:22.518Z)

Perfect. Now let me verify my findings about the overlapping directories by creating a summary. Let me also double-check the `_FIELD_DIR` usage:

#### 49. user (2026-07-03T17:35:23.593Z)

```
30: _FIELD_DIR = {
230:     field_dir = _FIELD_DIR.get(field)
245:     field_dir = _FIELD_DIR.get(field)
307:     choices=sorted(_FIELD_DIR),
```

#### 50. assistant (2026-07-03T17:35:25.876Z)

Let me understand the complete flow to see if there are truly unnecessary abstractions. Let me trace through to see what happens when `resolve_manifest_paths` is called:

Actually, I need to look at the usage pattern more carefully. Let me check the test cases:

#### 51. user (2026-07-03T17:35:26.206Z)

```
22     def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23         workspace = tmp_path / "workspace"
24         repo = workspace / "badminton-highlight-indexer"
25         repo.mkdir(parents=True)
26         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27         old_label = _write(
28             workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29         )
30         output_label = _write(repo / "output" / "Bar.rallies.csv")
31         manifest = _write_manifest(
32             repo,
33             [
34                 {
35                     "name": "foo",
36                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
37                     "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
38                     "fps": 30,
39                     "frame_width": 1920,
40                 },
41                 {
42                     "name": "bar",
43                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
44                     "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
45                     "fps": 30,
46                     "frame_width": 1920,
47                 },
48             ],
49         )
50
51         specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
52
53         assert issues == []
54         assert Path(specs[0]["trajectory"]) == traj.resolve()
55         assert Path(specs[0]["labels"]) == old_label.resolve()
56         assert Path(specs[1]["labels"]) == output_label.resolve()
57
```

```

58
59 def test_strict_loader_reports_missing_required_files(tmp_path):
60     repo = tmp_path / "repo"
61     repo.mkdir()
62     manifest = _write_manifest(
63         repo,
64         [
65             {
66                 "name": "missing",
67                 "trajectory": "C:/old/missing_traj.csv",
68                 "labels": "C:/old/missing_rallies.csv",
69             }
70         ],
71     )
72
73     with pytest.raises(gm.ManifestPathError) as excinfo:
74         gm.load_resolved_manifest(manifest, repo_root_path=repo)
75
76     assert len(excinfo.value.issues) == 2
77     assert "file not found" in str(excinfo.value)
78

```

## 52. assistant (2026-07-03T17:35:31.591Z)

Now let me create my analysis. I've found these issues:

1. `**resolve_manifest_paths` is a thin wrapper\*\* - it's only used in tests, not in any consumer
2. `**resolve_manifest_specs` is only called by `resolve_manifest_paths`\*\* - not used directly by consumers either
3. `**resolve_existing_path` is only called internally\*\* by `resolve_manifest_specs`
4. `**ManifestIssue` dataclass is only used for error reporting\*\* in `resolve_manifest_specs`
5. `**extra_roots` and `repo_root_path` parameters\*\* - only used when explicitly passed; `repo_root_path` defaults to `repo_root()` anyway for most calls
6. `**required_fields`\*\* - only `rally_seq_eval.py` uses a non-default value
7. `**Overlapping directory generation`\*\* - `_candidate_dirs` and `_expand_root` both generate `root/"output"`
8. `**Stale docstring`\*\* - `test_served_gate_a_regression._golden_specs()` docstring still describes OLD semantics (per-file filtering)
9. `**Possible unused json import`\*\* - `calibrate_local.py` removed `json` but I should verify it's truly unused elsewhere

Let me verify the `json` import in the trainer harness:

## 53. user (2026-07-03T17:35:31.944Z)

```

1      """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3      python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4      --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6      Productionizes the rally_seq prototype per ADR-008:
7      - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8      honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9      the FINAL model on all videos.
10     - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11         <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12         <out>/<version>/manifest.json    (the reviewable contract: feature-schema version,
13                                         calibration, training lineage + GT hashes,
14                                         eval results, attestations, gate verdict)
15     - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16       (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17       (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18       undefined => no self-declared victory). The product side owns final admission

```



```

19         (backend/eval/regression.py); this harness never grades itself into serving.
20
21     Training may import backend.* (one-way wall); backend must never import training.*.
22     """
23
24     from __future__ import annotations
25
26     import argparse
27     import hashlib
28     import json
29     import os
30     import re
31     import subprocess
32     import time
33     from math import comb
34     from typing import Dict, List, Optional
35
36     import numpy as np
37
38     from backend.eval.classifier import LogisticRegression
39     from backend.eval.golden_manifest import load_resolved_manifest
40     from backend.eval.gt_loader import load_gt_intervals
41     from backend.eval.rally_seq_proto import (
42         _seg_f1,
43         labels_for,
44     )
45     from backend.eval.rally_seq_proto import (
46         run as lovo_run,
47     )
48     from backend.eval.regression import gt_hash
49     from backend.features.rally_seq import (
50         FEAT_NAMES,

```

#### 54. user (2026-07-03T17:35:35.040Z)

```

225:         floor = json.load(f)
244:         json.dump(payload, f, indent=2)
292:         json.dump(manifest, f, indent=2)

```

#### 55. assistant (2026-07-03T17:35:45.926Z)

Good, json is still used in training/gen0/harness.py. Now let me compile all my findings into a structured format.

Based on my analysis of PR #452, here are the simplification candidates identified by examining the diff:

```

```json
[
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 86,
    "summary": "resolve_manifest_paths is a 2-line wrapper that only calls load_manifest then resolve_manifest_specs; only used in tests, not by any consumer. Could inline or mark internal.",
    "failure_scenario": "Consumers only call load_resolved_manifest; the intermediate resolve_manifest_paths function adds cognitive load without providing a value for the public API"
  },
  {
    "file": "backend/eval/golden_manifest.py",
    "line": 172,
    "summary": "resolve_existing_path is only called internally by resolve_manifest_specs (line 157). Not used elsewhere; could be private (_resolve_existing_path).",
    "failure_scenario": "Exported as public but never imported or called outside the module; API

```

```

surface bloat"
},
{
  "file": "backend/eval/golden_manifest.py",
  "line": 237,
  "summary": "Both _candidate_dirs (line 237) and _expand_root (line 246) generate
root/'output' separately; _candidate_dirs duplicates the field-dir logic for the main repo root
(lines 235-237) that _expand_root handles for extra_roots (lines 246-254). Single comprehension
could unify.",
  "failure_scenario": "Maintenance burden: if a new search path pattern is added, both
functions must be updated"
},
{
  "file": "backend/eval/golden_manifest.py",
  "line": 36,
  "summary": "ManifestIssue dataclass with message() method is only used for error formatting
in ManifestPathError.__init__ (line 56). Could be inline strings in the issues list instead.",
  "failure_scenario": "Adding a new issue type or changing message format requires modifying
the dataclass; strings would be equally clear and simpler"
},
{
  "file": "tests/test_served_gate_a_regression.py",
  "line": 69,
  "summary": "Docstring for _golden_specs() still says 'manifest specs whose trajectory +
label files are all present' ? describes OLD semantics of per-file filtering. Now it just calls
load_resolved_manifest which does that internally. Docstring should reflect the new behavior.",
  "failure_scenario": "Misleading docstring; readers think the function is filtering when it
now delegates that work to load_resolved_manifest"
},
{
  "file": "backend/eval/golden_manifest.py",
  "line": 89,
  "summary": "repo_root_path parameter defaults to None and uses repo_root() as fallback (line
142); extra_roots defaults to empty tuple. No consumer passes repo_root_path outside CLI/tests.
Consider whether these knobs justify the parameter inflation.",
  "failure_scenario": "If only the CLI needs repo_root_path override, it could be handled
there without cluttering the public function signatures; 7 consumer call sites all ignore
extra_roots"
}
]
...

```